Accessibility Features — Implementation Reference

The accessibility system is entirely client-side. Settings live in `localStorage` and are applied immediately by writing CSS classes to `document.documentElement`.

Source file: `frontend/src/app/accessibility/page.tsx`

---

localStorage Schema

Key: `accessibilitySettings`
Format: JSON string

```
{
  "highContrast": false,
  "TTS": false,
  "fontSize": 100
}
```

Field	Type	Default	Description
`highContrast`	boolean	`false`	High contrast mode — adds `high-color-contrast` CSS class
`TTS`	boolean	`false`	Text-to-speech — adds `TTS-option-switch` CSS class
`fontSize`	number	`100`	Font scale percentage — drives the `--font-scale` CSS custom property

Read/write pattern in accessibility/page.tsx:

- On mount: `useEffect` reads `localStorage.getItem('accessibilitySettings')` and populates React state. `fontSize` defaults to `100` via `??` if absent.
- On any change: a second `useEffect` (dependencies: `[highContrast, TTS, fontSize, accessibilitySettingsIsLoaded]`) writes the current state back to localStorage. The `accessibilitySettingsIsLoaded` flag prevents an overwrite on the initial render before the saved value has been loaded.

High Contrast Mode

CSS class: `high-color-contrast` on `document.documentElement`

When `highContrast = true`, the class `high-color-contrast` is added to the `<html>` element. CSS rules in `globals.css` that use `.high-color-contrast` selectors override the default color scheme with high-contrast alternatives.

L50–64 Implementation in accessibility/page.tsx:

```
useEffect(() => {
  const targetDom = document.documentElement;
  if (highContrast) {
    targetDom.classList.add('high-color-contrast');    // L55
```

```
  } else {
    targetDom.classList.remove('high-color-contrast'); // L57
  }
  if (TTS) {
    targetDom.classList.add('TTS-option-switch');      // L60
  } else {
    targetDom.classList.remove('TTS-option-switch');   // L62
  }
}, [highContrast, TTS]);
```

Note: The effect runs on all four accessibility state changes (not just `highContrast`). This re-evaluates all DOM class changes whenever any setting changes.

Font Size Slider

Component: `FontSizeSlider` (L331 in components.tsx)
Rendered on: accessibility/page.tsx L102–106

The `FontSizeSlider` renders a range input with four snap positions: **75%, 100%, 125%, 150%**. The active tick label is highlighted in the brand red. Changing the slider updates `fontSize` state, which is immediately persisted to localStorage and applied via the `--font-scale` CSS custom property.

```
// L66–70 in accessibility/page.tsx — applies font scale to the document root
useEffect(() => {
  if (typeof document === 'undefined') return;
  document.documentElement.style.setProperty('--font-scale', `${fontSize}%`);
}, [fontSize]);
```

CSS mechanism (globals.css):

```
:root { --font-scale: 100%; }
html  { font-size: var(--font-scale); }
```

Because all button heights and other dimensions now use `rem` (not `px`), changing `--font-scale` scales the entire UI proportionally.

High Contrast overrides: `.high-color-contrast .font-size-slider` and related selectors override accent colors and label colors to match the high-contrast palette.

Props:

Prop	Type	Description
`value`	`number`	Current font scale percentage (75, 100, 125, or 150)
`onChange`	`(value: number) => void`	Callback called with the new numeric value on slider change

---

Text-to-Speech (TTS) Setting

CSS class: `TTS-option-switch` on `document.documentElement`

When `TTS = true`, the class `TTS-option-switch` is added to `<html>`. This can be used in CSS to show/hide TTS-related UI elements (like the `TTSButton`).

The actual TTS behavior (speaking text) is implemented in `route/page.tsx` and the `TTSButton` component. The setting here only persists the preference to localStorage.

See the **Text-to-Speech (TTS)** section in frontend-flow.md for the full implementation: `isTTSEnabled()` (L27–33), `speakIfEnabled()` (L44–51), `TTSButton` (L69–82), and render locations (L273, L333, L401, L466).

How Settings Persist Across Pages

`localStorage` is per-origin (not per-page). Any page on the same origin can read `accessibilitySettings`.

- route/page.tsx L27–33 reads `TTS` directly via `isTTSEnabled()` (not through React state).
- Other pages apply High Contrast by reading the class on `document.documentElement` — since the class is added on the accessibility page and persists through navigation in Next.js (no full page reload), it stays active until removed.

Cold start (new tab or hard refresh): CSS classes and the `--font-scale` property are NOT applied automatically on other pages. Only `accessibility/page.tsx` writes them to `document.documentElement`. If a user has High Contrast enabled or a custom font size set and opens a new tab, neither will be re-applied until they visit the accessibility page again.

Planned: Accessibility Tracking

Future work (tracked in ROUTE_STATISTICS_SPEC.md §8) will send accessibility settings to the backend so staff can see usage rates in the portal. This requires:

1. A new `accessibilityFlags Json?` column on `PlayerSession`
2. A new endpoint `POST /api/set-accessibility-flags` (JWT-protected)
3. Calling the endpoint from `accessibility/page.tsx` on mount and on any toggle

Until this is implemented, accessibility usage is invisible to the portal.

Settings at a Glance

Stored key	localStorage key	Effect
`highContrast`	`accessibilitySettings`	Adds `high-color-contrast` class to `<html>`
`TTS`	`accessibilitySettings`	Adds `TTS-option-switch` class to `<html>`
`fontSize`	`accessibilitySettings`	Sets `--font-scale` CSS property on `<html>` (75–150%)